Plano de execução: navegação e experiência do cliente

Data: 29/10/2026. Estado: planejamento; não iniciar novas alterações de código até a próxima autorização do usuário. Este documento cobre as telas e jornadas do **cliente**. As áreas administrativa, do barbeiro e da atendente não fazem parte do redesign, salvo componentes comprovadamente compartilhados, que devem preservar o comportamento dos demais perfis.

1. Objetivo

Criar uma experiência coerente, moderna e fácil de usar no celular e no computador, permitindo que uma pessoa descubra os serviços, escolha um horário, entre na conta, confirme a reserva e depois encontre ou altere seus agendamentos.

Prioridades: clareza da próxima ação, orientação sobre onde o usuário está, prevenção de erros, recuperação sem perda de escolhas, acessibilidade e consistência visual. O estilo escuro e dourado da marca é o ponto de partida; cores, tipografia e imagens devem ser conferidas na aplicação real antes de se fixar uma solução visual.

Regras já definidas

- O cliente comum pode explorar serviços e iniciar o fluxo sem sessão, mas **precisa entrar ou cadastrar-se para confirmar**. A reserva só existe após sucesso do servidor.
- A atendente autorizada pode agendar manualmente para um contato sem conta; não criar esse privilégio para usuários comuns.
- Mostrar preço e duração cadastrados, disponibilidade real e situação verdadeira. Não prometer descontos, assinatura, lembrete, pagamento ou prazo que não estejam disponíveis.
- O pagamento continua sendo registrado pela recepção ou administração após conclusão, conforme as regras atuais. A interface de reserva não deve sugerir processamento de pagamento online.
- Alterações de interface não podem enfraquecer autorização, validação de horários ou preservar escolhas inseguras de forma indefinida.

2. Referências e limites da análise

Foram usadas as orientações das skills ui-pattern, ui-ux-designer e uxui-principles, além do plano existente docs/refatoracao-frontend.md. O arquivo CLAUDE.md remete a AGENTS.md. Antes de escrever código, a próxima execução deve ler a documentação pertinente em node_modules/next/dist/docs/ e verificar instruções adicionais eventualmente existentes.

A skill ui-pattern orienta compor padrões reutilizando componentes disponíveis; seus exemplos de classes não são um sistema de design comprovadamente existente neste repositório. Primeiro inventariar componentes e tokens reais, sem presumir que components/ui e components/patterns existem. A skill uxui-principles lista ferramentas de avaliação opcionais; este plano aplica os princípios descritos, sem alegar uma auditoria automatizada dos 168 itens ou instalar pacotes.

A inspeção para este plano foi estática, complementada pelo histórico de trabalho anterior. Não é um teste de usabilidade com clientes nem uma validação completa em produção. A apresentação comercial e capturas anteriores podem orientar a comparação, mas podem ter sido geradas em outra versão.

3. Ponto de partida: alterações locais já feitas

As mudanças abaixo constam no diretório de trabalho e devem ser preservadas e revisadas, não recriadas ou revertidas automaticamente:

Área	Situação observada	Próximo passo
src/components/lux/BottomNavigation.tsx	Rótulos visíveis, ação central destacada, indicação de página e foco para teclado.	Validar área segura, estado ativo, comportamento do menu e histórico do navegador.
ClientLuxDashboard.tsx, HeroBanner.tsx, QuickAccess.tsx	Ajustes no CTA, atalhos e duplicação de navegação; remoção de afirmações do hero sobre reserva.	Rever hierarquia, ações repetidas e primeira dobra, para visitante e cliente com reserva.
Header.tsx, SideDrawerMenu.tsx	Ajustes de menu, notificações, acesso por teclado e link de indicação baseado na origem.	Testar foco, fechamento, notificações, sessão, privacidade e viewport estreito.
BookingFlow.tsx	Erro de disponibilidade com nova tentativa, cancelamento da requisição anterior, estados acessíveis e fuso na confirmação.	Testar corridas de rede, datas, login/cadastro, conflitos e todas as etapas.
/agenda e /historico	Títulos, orientações, estados vazios e ações atualizados.	Padronizar cartões, navegação, paginação e dados apresentados.
/agenda/[id]	Falta e horário passado pendente não são apresentados como concluídos; mensagens de status foram ajustadas.	Validar todos os estados, cancelamento e reagendamento.
/auth/login, /auth/register, /profile	Ajustes de textos, avisos e controle de senha.	Completar a revisão visual e testar continuidade após autenticação.

Na última execução, foram informados 178 testes aprovados e verificação de tipos aprovada. O lint dos arquivos da última etapa não teve erro, com um aviso existente de uso de <img> na tela de detalhes. **Esses resultados não substituem nova execução nem comprovam navegação visual, acessibilidade ou integração com banco.**

Riscos e hipóteses verificadas no código

1. BookingFlow.tsx trabalha com uma lista de sete dias. Avaliar se é suficiente para a operação e especificar navegação acessível para datas além desse intervalo, respeitando regras existentes do servidor.
2. O estado de reserva em src/store/booking-store.ts usa armazenamento local e restaura escolhas, mas a continuidade por login e cadastro deve ser testada de ponta a ponta, inclusive com recarregamento e sessão Google quando configurada.
3. Em PremiumBanner.tsx há uma oferta com “cortes ilimitados”; ClientLuxDashboard.tsx abre contato de WhatsApp com número fixo. Antes de exibir oferta ou contato, confirmar dados e benefícios reais para cada barbearia. Esconder ou tornar neutro o que não puder ser comprovado.
4. A navegação inferior e o cabeçalho podem abrir menus por caminhos distintos; revisar quem controla o estado para evitar menus duplicados, foco perdido ou item ativo incorreto.
5. O menu lateral declara semântica de diálogo e fecha por Escape. Verificar efetivamente contenção e restauração de foco, bloqueio de rolagem e interação com leitores de tela.

6. O perfil é compartilhado com outros papéis. Qualquer mudança ali exige verificar também administrador, barbeiro e atendente.
7. ServicesCarousel.tsx já possui mensagem para catálogo vazio. Revalidar seu estado atual e evitar reintroduzir serviços fictícios; verificar cartões e imagens sem assumir que o documento antigo descreve o código atual.

4. Jornada desejada e arquitetura de navegação

Entrada principal: Início → conhecer serviço → escolher profissional → escolher data e horário → revisar → entrar/cadastrar se necessário → revisar novamente com dados preservados → confirmar → sucesso → consultar agenda.

Jornadas paralelas: Início → catálogo; Agenda → detalhes → reagendar/cancelar; Histórico → detalhes; Conta → dados pessoais/senha; Notificações → consulta e leitura.

Destino	Ponto atual / responsabilidade	Resultado esperado
Início /	BookingFlow.tsx e ClientLuxDashboard.tsx	Próxima ação clara, serviços acessíveis e próxima reserva quando houver.
Catálogo	ServicesCarousel.tsx e ServiceCard.tsx	Consultar preço e duração antes da escolha; “Ver todos” com destino e comportamento explícitos.
Agendamento	BookingFlow.tsx, booking-store.ts, API de disponibilidade	Progresso fiel, escolhas editáveis, disponibilidade e confirmação seguras.
Login/cadastro	src/app/auth/login e register	Autenticação compreensível e retorno ao contexto de origem.
Agenda	src/app/agenda/page.tsx	Próximas reservas, estado vazio e acesso aos detalhes.
Detalhes	src/app/agenda/[id]/AppointmentDetailClient.tsx	Situação, informações e ações elegíveis de reagendamento/cancelamento.
Histórico	src/app/historico/page.tsx	Atendimentos anteriores, situações corretas e nova reserva.
Conta	src/app/profile/page.tsx	Perfil e senha claros, com feedback e navegação adequada ao papel.
Navegação e notificações	Header, BottomNavigation, SideDrawerMenu	Mesmo vocabulário, destino inequívoco e controles acessíveis em todas as telas.

O catálogo pode ser uma seção navegável ou uma página dedicada. Decisão técnica futura: verificar se é possível compartilhar catálogo e seleção sem duplicar dados ou criar uma rota desnecessária. Em qualquer opção, o usuário precisa conseguir explorar antes de iniciar a reserva.

5. Sistema visual e padrões compartilhados

1. Inventariar cores em src/app/globals.css, classes existentes e componentes compartilhados. Definir tokens semânticos para fundo, superfície, texto, borda, destaque, sucesso, alerta e erro, preservando o tema de outros perfis.

2. Unificar botão primário, secundário e discreto; tamanho confortável para toque, estados hover/foco/pressionado/desabilitado e carregamento. Não depender apenas de cor para transmitir seleção ou erro.
3. Padronizar cabeçalhos de tela, cartões de serviço e agendamento, etiquetas de situação, formulários, campos, mensagens e estados vazios. Reutilizar componentes existentes quando suportarem o comportamento necessário.
4. Usar uma hierarquia de fonte coerente, textos descritivos, alinhamento consistente e margens adequadas. Evitar rótulos cortados, ícones sem nome e tamanhos de texto excessivamente pequenos.
5. Utilizar fotografias adequadas ao serviço, com proporção consistente e alternativa para imagem ausente. Evitar associar uma foto genérica a um serviço específico sem contexto.
6. Respeitar preferência por movimento reduzido; animações devem reforçar a mudança de estado, sem impedir a navegação ou ocultar conteúdo.
7. Definir comportamentos de carregamento, atualização, vazio, erro e sucesso para cada padrão. Manter contraste verificável e texto legível em aumento de fonte.

Padrões a compor

- **Cabeçalho de página:** título, descrição breve e ação contextual opcional.
- **Cartão de serviço:** nome, duração, preço válido e ação; sem truncar informação essencial.
- **Cartão de reserva:** serviço, data, hora, profissional, situação textual e entrada em detalhes.
- **Grupo de seleção:** controles nomeados, seleção perceptível e explicação para indisponibilidade.
- **Estado de feedback:** mensagem + causa compreensível + ação possível, sem atribuir uma falha técnica à ausência de dados.
- **Diálogo/folha móvel:** título, foco inicial, Escape quando aplicável, foco contido, retorno ao gatilho e área segura.

6. Plano por fases

Fase 0 — Diagnóstico, segurança e linha de base

Ações

- Ler o diff local, documentos existentes e regras de Next.js; preservar arquivos não relacionados e mudanças em andamento.
- Inventariar rotas, entradas de agendamento, componentes, estados, cópias e telas compartilhadas entre perfis.
- Levantar em ambiente de teste as jornadas com e sem sessão; capturar mobile e desktop, estados vazios, erro e carregamento.
- Registrar suposições de negócio: endereço/contato real, ofertas, política de cancelamento, serviços ativos e janela de agendamento. Não inventar essas informações.
- Com autorização e dados de teste, observar pelo menos três pessoas tentando agendar, consultar e reagendar. Se não houver participantes, registrar que a validação ficou pendente, sem inventar resultados.

Aceite: mapa de jornada e problemas priorizados; telas existentes e mudanças locais identificadas; ambiente seguro e dados de teste preparados sem modificar o banco de produção.

Fase 1 — Navegação, cabeçalho, menu e notificações

Arquivos principais: Header.tsx, BottomNavigation.tsx, SideDrawerMenu.tsx, ClientLuxDashboard.tsx.

- Definir as entradas Início, Agenda, Agendar, Histórico e Conta/Mais, com nomes consistentes em desktop e mobile.
- Usar semântica de link para navegação e botão para mudança de estado, quando adequado. O estado ativo deve acompanhar a rota e a etapa, inclusive após voltar/avançar.
- Centralizar o controle do menu e das notificações; impedir abertura dupla, preservar e restaurar foco e permitir fechamento por teclado.
- Corrigir sobreposição de conteúdo com barra inferior, teclado virtual e áreas seguras; verificar Safari e Chrome em aparelhos reais quando disponíveis.
- Identificar notificações não lidas e permitir marcá-las como lidas por teclado; tratar falhas de leitura sem afirmar sucesso indevido.
- Revisar compartilhamento/indicação para usar endereço válido e contexto real, sem divulgar benefícios não existentes.

Aceite: cada destino abre uma única vez; o item ativo corresponde à tela; menus funcionam por toque e teclado; conteúdo e última ação não ficam escondidos.

Fase 2 — Início, serviços e conteúdo confiável

Arquivos principais: HeroBanner.tsx, QuickAccess.tsx, AppointmentCard.tsx, ServicesCarousel.tsx, ServiceCard.tsx, PremiumBanner.tsx.

- Compactar a primeira dobra e oferecer CTA claro. Quando houver próxima reserva, dar acesso evidente a ela sem ocultar serviços.
- Diferenciar “ver serviços” de “iniciar reserva”; escolher serviço pode preencher a etapa correspondente.
- Exibir nome completo, duração e preço cadastrado. Catálogo vazio informa a situação, sem exemplos fictícios.
- Revisar oferta de assinatura e contato fixo: só apresentar se houver oferta e canais confirmados/configuráveis. Caso contrário, ocultar ou substituir por conteúdo verdadeiro aprovado.
- Usar imagens coerentes e fallback; garantir carrossel navegável com teclado e indicação de que existem mais itens.
- Conferir o cartão de próxima reserva com vários, nenhum ou agendamentos antigos; não inferir profissional a partir do primeiro da lista.

Aceite: visitante identifica serviços e ação de reservar; cliente encontra seu próximo horário; não há preço, oferta, endereço ou contato inventado.

Fase 3 — Fluxo de agendamento

Arquivos principais: BookingFlow.tsx, src/store/booking-store.ts, src/app/api/barbers/[id]/availability/route.ts e validações de criação existentes.

Serviço e profissional

- Apresentar progresso real e escolha atual; identificar claramente valor e duração.
- Se a pessoa alterar serviço ou profissional, mostrar e explicar a eventual necessidade de escolher novamente o horário; manter a lógica de invalidação das escolhas dependentes.

- Tratar catálogo ou equipe vazios com orientação, sem exemplos fictícios.

Data e horário

- Estender ou esclarecer a navegação além dos sete dias quando a regra da operação permitir; não mostrar datas fora do limite aceito pelo servidor.
- Mostrar data completa, dia da semana, seleção e informações sobre indisponibilidade. Garantir navegação por teclado, toque e rolagem sem depender apenas de arrastar.
- Diferenciar carregando, nenhum horário, falha de rede, sessão expirada quando aplicável e conflito. Preservar a tentativa novamente e abortar consultas anteriores.
- Ao receber conflito no envio, preservar serviço/profissional, explicar que o horário deixou de estar disponível e oferecer nova seleção. Verificar que o aviso apareça mesmo após a troca de etapa.
- Centralizar o tratamento do fuso da barbearia: testar usuário em outro fuso, meia-noite e mudança de dia. Não confundir datas locais do navegador com datas do negócio.

Revisão, autenticação e sucesso

- Mostrar serviço, profissional, data completa, duração, preço, condição de pagamento e política de cancelamento somente quando confirmada pela regra real.
- Permitir editar cada item com nomes acessíveis específicos, como “Editar serviço”, preservando escolhas válidas.
- Informar “ainda não confirmado” até a resposta de sucesso. Desabilitar envio duplicado e mostrar progresso da confirmação.
- Cliente sem sessão segue para login ou cadastro; manter escolhas e retornar à revisão após autenticação, sem criar a reserva automaticamente. Testar validade do rascunho antigo e revalidar preço, serviço ativo e horário antes de confirmar.
- Na confirmação, mostrar resumo e caminhos para Agenda e Início. Em erro, oferecer ação específica sem apagar a seleção sem motivo.

Aceite: nenhuma reserva é criada sem autenticação; reserva só é confirmada pelo servidor; falha e conflito são distintos; data/horário permanecem corretos; não há confirmação duplicada.

Fase 4 — Agenda, detalhes, histórico e conta

Arquivos principais: src/app/agenda/page.tsx, src/app/agenda/[id]/AppointmentDetailClient.tsx, src/app/historico/page.tsx, src/app/profile/page.tsx.

- Agenda: apresentar próximas reservas em ordem, informações essenciais, paginação compreensível e CTA contextual. Distinguir “sem registros” de página sem resultados e falha de carregamento.
- Detalhes: mostrar estado verdadeiro, valor, profissional e data; exibir ações apenas quando elegíveis e explicar por que não estão disponíveis quando necessário.
- Cancelamento: apresentar consequências reais e pedir confirmação; cancelar o diálogo não altera a reserva. Após sucesso, atualizar tela, agenda e histórico.
- Reagendamento: escolher data e horário disponíveis, mostrar antes/depois, confirmar e informar conflitos. Revalidar no servidor e não tratar resposta antiga como disponibilidade atual.
- Histórico: rótulos compreensíveis para concluído, cancelado, falta e situação pendente; acesso aos detalhes quando disponível e nova reserva.

- Conta: agrupar dados pessoais e segurança, rótulos claros, feedback de salvamento, alterações pendentes e tratamento de foto indisponível. Verificar que ajustes não prejudiquem os outros perfis.

Aceite: as ações apresentadas correspondem às permissões e ao estado; usuário encontra a reserva, executa uma alteração e recebe resultado inequívoco.

Fase 5 — Login e cadastro

Arquivos principais: src/app/auth/login/page.tsx, src/app/auth/register/page.tsx, src/core/utils/safe-redirect.ts.

- Clarear por que a conta é necessária e para onde o usuário voltará; manter valores e callback sanitizado.
- Revisar rótulos, autocomplete, exigências de senha reais, foco, botões de exibir senha e mensagens de erro/sucesso.
- Preservar o destino entre login e cadastro; testar conta com senha e Google somente quando o provedor estiver configurado.
- Em falha, preservar os dados que podem ser mantidos com segurança e indicar o próximo passo. Não expor credenciais em URLs, logs ou análises.
- Não prometer recuperação de senha, métodos de autenticação ou envio de mensagens que não existam na aplicação.

Aceite: visitante consegue autenticar e voltar à revisão sem perder escolhas válidas; callback externo ou inválido é rejeitado; cadastro não confirma reserva por conta própria.

Fase 6 — Revisão transversal e entrega

- Conferir desktop e mobile, responsividade, contraste, zoom, foco e movimento reduzido em todos os pontos da jornada.
- Executar testes existentes e acrescentar testes significativos para regressões e fluxos alterados. Não testar apenas classes de estilo.
- Validar em navegador com dados de teste isolados: visitante, cliente autenticado, agenda vazia/preenchida, indisponibilidade e conflito.
- Revisar cópias, comportamento ao voltar/recarregar, URL, persistência, permissões e componentes compartilhados com outros perfis.
- Registrar o que foi validado, os problemas restantes e capturas antes/depois; não afirmar conformidade de acessibilidade ou ganhos de conversão sem avaliação adequada.

Aceite: jornada crítica completa sem bloqueios conhecidos; estilos e padrões coerentes; resultados de validação e limitações documentados.

7. Matriz de validação obrigatória

Cenário	Resultado esperado
Visitante escolhe serviço, profissional e horário	Resumo informa que a reserva ainda não foi confirmada.
Visitante entra ou se cadastra	Retorna à revisão com escolhas válidas; sem gravação automática.
Mudança de serviço ou barbeiro	Horário incompatível é invalidado e o usuário entende o que precisa selecionar de novo.
Falha na consulta de horários	Mensagem de falha e opção de tentar novamente; não mostrar “sem horários” como causa.
Troca rápida de datas	Resultado antigo não sobrescreve o dia selecionado.
Horário ocupado antes da confirmação	Mensagem de conflito visível, sem reserva duplicada; oferecer alternativas.
Dois cliques ou rede lenta ao confirmar	Controle de envio e verificação no servidor evitam dupla reserva.
Atendimento cancelado, falta, concluído ou pendente passado	Situação apresentada corresponde ao estado real.
Cancelamento/reagendamento negado pela regra	Explicação compreensível, sem mudar a situação no frontend.
Navegação por teclado e leitor de tela	Foco visível, ordem lógica, nome, estado e feedback anunciados; menus e diálogos devolvem o foco.
Tela 320–390 px e zoom de texto	Conteúdo legível; ações acessíveis; sem elementos sobrepostos ou rolagem horizontal da página.
Recarregar e voltar/avançar	Navegação e estado são coerentes, sem reserva fantasma e sem perder escolhas válidas injustificadamente.
Administrador, barbeiro ou atendente no perfil compartilhado	Navegação e permissões do respectivo perfil continuam funcionando.
Serviço desativado ou preço alterado durante login	Revalidar com a fonte atual e esclarecer a alteração antes da confirmação.

Na implementação, rodar npm run typecheck, lint adequado, testes unitários pertinentes e testes de integração/navegação com os serviços necessários. Se o diretório temporário do ambiente causar erro de permissão, configurar TEMP/TMP para uma pasta de trabalho permitida, sem alterar arquivos do usuário fora do projeto. Registrar separadamente qualquer teste de banco/Google/Safari não executado.

8. Métricas e pesquisa após implantação

Usar somente dados necessários e autorização apropriada. Medir como hipóteses, não promessas: taxa de confirmação por jornada iniciada, abandono por etapa, retorno ao fluxo após login, conflitos por horário, erros de disponibilidade e tempo para concluir tarefas. Segmentar por dispositivo quando tecnicamente permitido. Não enviar nome, telefone, e-mail ou dados financeiros individuais a ferramenta externa de análise por padrão.

Comparar com uma linha de base do mesmo período e composição de público. Convidar clientes e atendentes a relatar onde houve dúvida. Ajustar a interface com base nos resultados; mudanças visuais isoladas não provam aumento de conversão.

9. Ordem de entrega e definição de pronto

1. Fase 0: confirmar estado atual e regras de conteúdo.
2. Fase 1: navegação e padrões compartilhados.
3. Fase 2: início e catálogo.
4. Fases 3 e 5: agendamento e autenticação tratados como uma única jornada integrada.
5. Fase 4: agenda, detalhes, histórico e conta.
6. Fase 6: validação transversal e correção de regressões.

Considerar o trabalho concluído quando as fases forem implementadas e verificadas, o cliente conseguir percorrer as jornadas principais sem ajuda e todas as limitações de teste forem registradas. As melhorias já feitas são parte do trabalho, não evidência de conclusão do conjunto.